

# LAB1

## Prise en main de l'environnement



### Table des matières

|                                                     |   |
|-----------------------------------------------------|---|
| 1 Objectifs.....                                    | 2 |
| 2 Rappels et mise en place de l'environnement.....  | 2 |
| 3 Travaux préparatoires.....                        | 2 |
| 4 Connaissance système.....                         | 2 |
| 5 Source, code assembleur, binaire et débogage..... | 3 |
| 6 Évolution de code.....                            | 4 |
| 7 Amélioration de la fiche « débogage ».....        | 5 |



# 1 Objectifs

Ce LAB permet de mettre prendre en main l'environnement minimal pour aborder les notions de programmation système.

- Documents :
  - Supports : « Linux – Les commandes » et « Linux – L'éditeur Vi » ;
  - Fiches : « Compilation » et « Débogage ».
- Notions abordées : l'environnement de développement, le débogage.
- Commandes et fichiers exploités : gcc, commandes de gestion de fichiers, ddd.
- Travail à rendre : Vous devrez répondre directement à plusieurs questions au sein de ce document. Vous le copierez sur Moodle sous le nom : « LAB0\_NOM.pdf ».

**Pour développer, vous devez utiliser les outils standards GNU (gcc + gdb/ddd) et un éditeur de texte type Vi/Emacs/Gedit, l'utilisation d'un IDE est interdite.**

# 2 Travaux préparatoires

Pour vous familiariser à nouveau avec l'environnement :

- Lisez et effectuez les commandes décrites aux chapitres §1 à §3.7 du support « Linux - Les commandes.pdf » ;

# 3 Connaissance système

Objectif : Connaître le système sur lequel vous êtes en train de travailler : nom de la distribution, version du noyau, capacité de la machine (CPU, mémoire, carte mère et bios, cartes additionnelles), topologie du disque (marque, taille, partitionnement) ?

- Copiez ci-dessous les informations recherchées (les commandes utilisées et le résultat de celles-ci pour votre système).

CPU : `cat /proc/cpuinfo` Version: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz

Taille mémoire vive : `free` total : 16174628

Carte mère et version du BIOS : `dmidecode -t bios` Version: 1.14.0

Cartes additionnelles : `lshw | grep « description`

description: VGA compatible controller

description: Audio device

description: USB controller

description: Serial bus controller

Partitionnement du disque dur : `fdisk -l`

| Périphérique | Début | Fin | Secteurs | Taille | Type |
|--------------|-------|-----|----------|--------|------|
|--------------|-------|-----|----------|--------|------|

|           |      |            |            |        |                           |
|-----------|------|------------|------------|--------|---------------------------|
| /dev/sda1 | 2048 | 1638402047 | 1638400000 | 781,3G | Données de base Microsoft |
|-----------|------|------------|------------|--------|---------------------------|

|           |            |            |           |        |                           |
|-----------|------------|------------|-----------|--------|---------------------------|
| /dev/sda2 | 1638402048 | 1953523711 | 315121664 | 150,3G | Système de fichiers Linux |
|-----------|------------|------------|-----------|--------|---------------------------|

- Comment lire le journal de démarrage du système (boot) ?
- Comment lire de manière continue le journal d'événement ?

Commande :

afficher le journal de démarrage :

`journalctl -b`

lire de manière continue le journal d'événement :

`journalctl -f`

- Ouvrez un terminal.

Dans quel répertoire vous trouvez-vous ?

Commande :

`pwd`

`/home/victor`

- Dans votre répertoire de connexion, créez un répertoire `tmp`
- Positionnez les droits d'accès à `rwX r-x ---` pour `tmp`
- Copiez les fichiers `passwd`, `group` et `hosts` du répertoire `/etc` dans `tmp`
- Changez le nom de `hosts` en `hotes`.
- Positionnez les droits d'accès à `rw- r- - - - -` pour le fichier `hotes`. Lisez le contenu de `hotes`.  
Remarque : la lecture du fichier `~/tmp/hotes` est permise. Le fichier peut néanmoins être vide.
- Retirez au propriétaire le droit en lecture sur le fichier `hotes` et essayez de le lire.

Quel est la signification du message d'erreur obtenu ?

cat: `hotes`: Permission non accordée

il ne veut pas que l'on lise le fichier

- Remettez pour le propriétaire le droit en lecture sur le fichier `hotes`.
- Retirez pour le propriétaire le droit en écriture sur le répertoire `tmp`.
- Essayez de détruire `hotes`.

Quel est la signification du message d'erreur obtenu ?

`rm` : supprimer '`hotes`' qui est protégé en écriture et est du type « fichier » ?

il ne veut pas que l'on supprime le fichier

- Retirez pour le propriétaire le droit en lecture sur le répertoire `tmp`.
- Essayez de lister le contenu de `tmp`.

Quel est la signification du message d'erreur obtenu ?

`ls`: impossible d'ouvrir le répertoire '`tmp2`': Permission non accordée

il ne veut pas que l'on lise le contenu du repertoire

- Lisez le contenu de `hotes`.

Pourquoi pouvez-vous le lire ?

Parce ce qu'il y a le droit d'exécution

- Retirez pour le propriétaire le droit en exécution sur le repertoire `tmp`.
- Essayez de vous positionner sur ce repertoire.
- Quel est la signification du message d'erreur obtenu ?

bash: cd: tmp2: Permission non accordée

il ne veut pas entrer dans le repertoire

- Essayez de lire le contenu de `hotes`.

Quel est la signification du message d'erreur obtenu ?

cat: tmp2/hotes: Permission non accordée

il ne veut pas lire le contenu du fichier `hotes`

#### 4 Source, code assembleur, binaire et débogage.

- Lisez la fiche « rappel compilation » ;
- Réalisez le premier programme de la fiche `welcome.c` ;
- Lancez et analysez la compilation étape par étape comme décrite dans la fiche « rappel compilation ».

Copier ici, le **code assembleur** de ce programme.

```
.file "welcome.c"
.text
.section .rodata
.LC0:
.string "Rexy is welcome you"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
```

```

    movq %rsp, %rbp
    .cfi_def_cfa_register 6
    leaq .LC0(%rip), %rax
    movq %rax, %rdi
    call puts@PLT
    movl $0, %eax
    popq %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
.LFE0:
    .size main, .-main
    .ident "GCC: (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0"
    .section .note.GNU-stack,"",@progbits
    .section .note.gnu.property,"a"
    .align 8
    .long 1f - 0f
    .long 4f - 1f
    .long 5
0:
    .string "GNU"
1:
    .align 8
    .long 0xc0000002
    .long 3f - 2f
2:
    .long 0x3
3:
    .align 8
4:

```

Pour connaître le type d'un fichier (binaire, son, image, etc.), on peut se fier à son extension (.mp3, .jpeg, etc.). Cela est limité surtout quand le nom du fichier ne possède pas d'extension. La commande `file` sous Linux permet de connaître le type d'un fichier en faisant abstraction de son nom.

Quel type de fichier votre compilateur a-t-il généré ? Récupérez un fichier binaire exécutable pour Windows (un .exe quelconque) et testez la commande `file` avec celui-ci ?

Type du fichier binaire Linux :

welcome: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter

/lib64/ld-linux-x86-64.so.2, BuildID[sha1]=7e123401f26e4f6fa83611b5e05a824ac2f864d6, for  
GNU/Linux 3.2.0, not stripped

Type du fichier binaire Windows :

DiscordSetup.exe: PE32 executable (GUI) Intel 80386, for MS Windows

- Analysez et écrivez le deuxième programme `sentence2words.c` ;
- Lancez-le pas à pas dans le débogueur (*ddd*, *gdb* ou autre) pour suivre l'évolution du chargement du tableau de mots.

Réalisez une copie d'écran du débogueur affichant la zone mémoire du tableau rempli. Dans cette copie d'écran, on doit aussi voir la fenêtre de résultat du programme affichant le tableau de mots.

The screenshot shows a debugger interface with three main components:

- Memory Window (Left):** Displays the contents of the `words` array. It shows six strings: `"a\000\000\000\000\000\000\000\001"`, `"little\000UU"`, `"step\000(\376\367\377\177"`, `"for\000\000\000\000\000\000"`, `"the\000\377\177\000\000\000\020"`, and `"man\000\000\000\000\000\000\001"`.
- Register Window (Right):** Shows the `%x` register pointing to the memory address `0x61`.
- Code Window (Bottom Left):** Displays the C code for `sentence2words.c`. The code is as follows:

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     char sentence[] = "a little step for the nan";
6     char words[6][10];
7     int i=0,j=0,k=0;
8     do
9     {
10        if(sentence[i]!=' ')
11            words[j][k++] = sentence[i];
12        else
13        {
14            words[j++][k]='\0';
15            k=0;
16        }
17    } while(sentence[i++]!='\0');
18    for(i=0;i<6;i++)
19        printf("%s\n",words[i]);
20
21    return 0;
22 }
```
- Output Window (Bottom Right):** Shows the output of the program, displaying the words: `a`, `little`, `step`, `for`, `the`, and `man`.

## 5 Évolution de code

Proposez l'évolution suivante du code du deuxième programme :

- Au lancement du programme, la phrase initiale est demandée à l'utilisateur ;

- Le tableau est dimensionné dynamiquement en fonction de cette phrase ;
- L’affichage du tableau est réalisé par l’appel d’une fonction.

Copier votre code sur Moodle sous le nom de fichier « LAB1\_NOM.c ».

**Rappel : un fichier source qui ne compile pas n'est pas corrigé !**

Copiez ci-dessous une copie d’écran du résultat de son exécution

```
victor@victor-G5-5590:~/TP/A4S7/systeme/TD1$ gcc -o phrase LAB1_BOUVIER_D_ACHER.c -g -Wall
victor@victor-G5-5590:~/TP/A4S7/systeme/TD1$ ./phrase
Ecrire une phrase
voici une phrase pour monsieur l'encadrant
voici
une
phrase
pour
monsieur
l'encadrant
```

## 6 Amélioration de la fiche « débogage »

L’objectif est d’améliorer cette fiche, exploitez ddd sur un programme utilisant des fonctions et expliquez le rôle des fichiers core.

Copiez votre fiche « débogage » sur Moodle sous le nom « debug\_NOM.pdf »

**Vérifiez que vous avez bien copié 3 fichiers sur Moodle : ce LAB, le code de l'évolution du programme sentence2words.c et votre fiche débogage.**